



Amrita School of AI
COIMBATORE CAMPUS

SESSIONS 9-13

Kubernetes

You stop saying *how*, and start saying *what should be true*.

Dr. Abhijith Anandakrishnan · Assistant Professor
24AIM332 · Odd Semester 2026-27

Compose runs containers on a machine.

Kubernetes runs containers on a **fleet**, and keeps them running when machines die.

Declarative, not imperative

Imperative "Start three copies. If one dies, notice and start another."

You own the loop.

Declarative "There should be three copies."

A **controller** owns the loop, forever, comparing desired state to actual state.

RECONCILIATION

Control plane and nodes

CONTROL PLANE

- └─ api-server the only way in
- └─ etcd the desired state
- └─ scheduler picks a node
- └─ controllers reconcile

NODES

- └─ kubelet runs containers
- └─ kube-proxy networking

Everything talks to the **api-server**. Nothing talks to etcd directly.

Controllers watch, compare, and act. That loop is the whole system.

OBJECTS

The five you need on day one

OBJECT	IS
Pod	one or more containers, scheduled together, sharing an IP
Deployment	keeps N pods running, handles rollouts
Service	a stable address in front of changing pods

OBJECT	IS
Ingress	HTTP routing from outside the cluster
ConfigMap / Secret	configuration and credentials, separate from the image

You almost never create a Pod directly.

You declare a Deployment, and it creates Pods for you.

Three replicas, declared

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: myapp:1.4.2
          ports: [{ containerPort: 80 }]
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 500m, memory: 512Mi }
```

requests and limits

requests — what the scheduler reserves. It decides **which node** you land on.

limits — the hard ceiling. Exceed memory and the pod is killed (OOMKilled).

No requests → the scheduler assumes you need nothing → it packs the node → everything degrades together. This is the single most common cause of a mysteriously unhealthy cluster.

Pods come and go. The Service address does not.

```
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web }      # anything with this label
  ports: [{ port: 80, targetPort: 80 }]
```

Selectors match on **labels**, not names. That indirection is what makes rolling updates invisible to callers.

How far out does it reach

TYPE	REACH
ClusterIP	inside only (default)
NodePort	a port on every node
LoadBalancer	a cloud load balancer

For HTTP, usually none of these directly.

You use one `LoadBalancer` for an **Ingress controller**, and route by hostname and path behind it.

One load balancer, many services.

Two probes that mean different things

liveness — “are you alive?” Fails → the container is **restarted**.

readiness — “can you take traffic?” Fails → removed from the Service, **not** restarted.

Using a liveness probe that depends on the database means a database blip restarts every pod, which makes the outage worse. Liveness should test the process, not its dependencies.

Changing the image is the deployment

```
kubectl set image deploy/web \
  web=myapp:1.5.0
```

```
kubectl rollout status deploy/web
kubectl rollout undo  deploy/web
```

The Deployment brings up new pods, waits for **readiness**, then removes old ones.

If readiness never passes, the rollout stalls rather than taking the service down.

Manual, then automatic

```
kubectl scale deploy/web --replicas=10
```

```
kubectl autoscale deploy/web \  
  --min=3 --max=20 --cpu-percent=70
```

The autoscaler needs **requests** set, because it scales on a percentage *of the request*.

No requests → no autoscaling. The two features are linked.

kubectl, day one

```
kubectl apply -f deploy.yaml      # declare
kubectl get pods -o wide          # what exists, and where
kubectl describe pod web-abc123    # why is it unhappy (read Events)
kubectl logs -f deploy/web        # follow output
kubectl exec -it web-abc123 -- sh  # shell inside
kubectl get events --sort-by=.lastTimestamp
kubectl delete -f deploy.yaml
```

`describe` FIRST, THEN `logs` . MOST FAILURES ARE VISIBLE IN EVENTS.

Kubernetes is not always the answer

● **Worth it**

- Many services, many machines
- Real availability requirement
- A team that changes things often
- Autoscaling genuinely needed

● **Not worth it**

- One app on one server
- Two people, no ops budget
- A static site
- Docker Compose already works

A managed service you never see beats a cluster you cannot run.

Next: how the cluster and everything under it gets created in the first place.

Without anyone clicking in a console.